

1 Prompts for Data Synthesis and Inference

This part provides the core prompt templates used in our framework. First, we present the templates designed for CoT Synthesis, which instantiate the KGRS paradigm. These prompts are engineered to elicit structured, reproducible reasoning traces that are explicitly anchored to the provided knowledge context (schema, metrics, rules) to generate high-quality training data:

- **Natural-Language Question Generation with Emergent Augmentation** (Fig. 1): Transforms formal slot-filled intermediate representations into fluent, human-like business queries. This prompt encourages "emergent augmentation," allowing the model to naturally introduce analytical variations—such as ranking, trend analysis, or extremum selection—while ensuring all generated questions remain strictly grounded within the constraints of the provided schema and metric knowledge graphs.
- **Multi-Step Inference CoT Synthesis** (Fig. 2): Guides the decomposition of a complex analytical question and produces a detailed, first-person narrative explaining the stepwise logic and SQL design choices (e.g., CTEs, window functions).
- **Reflection CoT Synthesis** (Fig. 3): Generates a debugging trace by analyzing an erroneous SQL query and its error message to synthesize a correction path.
- **Degenerate Dimension CoT Synthesis** (Fig. 4 Fig. 5): Generates reasoning traces for adapting SQL queries to denormalized schemas where dimension attributes have been physically merged into fact tables, eliminating the need for JOIN operations while maintaining equivalent business logic.
- **Clarification CoT Synthesis** (Fig. 6): Produces reasoning to identify ambiguity in a user question and formulates a concise clarification request based on the available schema.
- **Dimension Rejection CoT Synthesis** (Fig. 7): Constructs a rationale for rejecting a query due to missing dimension fields in the schema.
- **Metric Rejection CoT Synthesis** (Fig. 8): Constructs a rationale for rejecting a query due to an undefined metric.
- **Follow-up CoT Synthesis** (Fig. 9): Generates reasoning to identify a missing mandatory constraint and asks the user for the required parameter.

In addition to the synthesis templates above, we provide distinct templates for validation and deployment. **SQL Validation** (Fig. 10) guides a stepwise analysis to judge if a SQL query correctly answers a business question. Finally, **Model Inference** (Fig. 11) serves as the unified template for deployment, integrating all knowledge sources in a few-shot format to guide the model’s <think> and <answer> generation.

Prompt for Natural-Language Question Generation with Emergent Augmentation

— 0. Background Knowledge Context

Schema Knowledge (Schema KG)

{schema_info}

Complete DDL definitions including fact tables, dimension tables, primary/foreign keys, and semantic field descriptions.

Metric Knowledge (Metric KG)

{metric_definitions}

Business metric definitions with aliases, mandatory constraints, and computation rules (e.g., "Net New Leads" requires is_net_leads=1 and count(distinct customer_account_id)).

Domain Knowledge (Domain KG)

{domain_rules}

Business rules for time range interpretation, grouping defaults, and analytical conventions (e.g., "last 7 days", default channel granularity).

Data Knowledge (Data KG)

{entity_mappings}

Entity-to-field mappings such as channel hierarchies and geographic attributes.

— 1. Role and Task

You are a business analyst fluent in domain terminology. Your task is to transform the given slot-filled structure into a natural, human-style business question that aligns with real-world BI scenarios.

— 2. Input: Slot-Filled Structure

{template_instance}

Example: [Time: Year 2025, Brand: Li Auto L6, Metric: Leads, GroupBy: By Month]

— 3. Generation Guidelines

(1) **Natural Expression:** Generate fluent questions as a business user would ask, not mechanical template translations.

(2) **Emergent Augmentation:** You may naturally introduce analytical variations that fit the business context, such as:

- *Ranking queries:* "Show me the top-3 months..."
- *Extremum selection:* "Which month had the highest..."
- *Direct aggregation:* "What is the total/average..."
- *Trend analysis:* "How did ... change over time..."

(3) **Knowledge Constraints:** All entities, metrics, and analytical operations must reference only elements defined in the provided Knowledge Context. Do not introduce undefined dimensions or metrics.

(4) **No Fixed Distribution:** Augmentation types should emerge naturally based on business relevance—do not force a predefined distribution.

— 4. Output Examples

Input: [Time: Year 2025, Brand: Li Auto L6, Metric: Leads, GroupBy: By Month]

Valid Outputs:

- "What is the monthly lead count for Li Auto L6 in 2025?" (*direct translation*)
- "In 2025, which month had the highest leads for Li Auto L6?" (*extremum*)
- "Show me the top-3 months by lead count for Li Auto L6 in 2025." (*ranking*)

— 5. Start Task

Please generate a natural-language question based on the slot-filled structure above. Output only the question text without additional explanations.

Figure 1: Prompt for Natural-Language Question Generation with Emergent Augmentation

Prompt for Multi-Step Inference COT Generation

— 0. Background Knowledge

Database Schema: {schema_info}

Knowledge Graph and Business Logic: {knowledge_info}

Domain Knowledge and Rules: {domain_info}

Supplementary Knowledge: {data_kg}

— 1. Role and Task

You are a senior data analytics architect who excels at decomposing complex multi-layered business problems into clear analytical logic and can deeply explain the construction approach and implementation details of complex SQL queries. Your task is to generate a detailed, in-depth, and logically coherent chain-of-thought (COT) reasoning process based on the context, business analysis question, and validated SQL query provided. This reasoning process should fully demonstrate your entire reasoning journey from understanding the problem to designing the SQL structure and then to implementation details.

— 2. Core Instructions

(1) First-Person Narrative: Your reasoning process should resemble a senior analyst's internal monologue when solving problems. Please narrate in first person ("I").

(2) Natural Analysis Flow: Your analysis should unfold naturally, starting from understanding the core requirements, complexity, and ultimate goals of the problem, then analyzing what data tables are needed and their associations, and finally explaining how filtering conditions, metric calculations, and grouping dimensions are determined based on business needs.

(3) Deep Explanation of Complex Logic: When SQL uses complex logic (such as CTEs, window functions, or multi-layer comparative calculations), you **must explain in depth why** you chose this technique and how it helps you construct the final result step by step. For example, when performing "above average" comparisons, you need to clearly explain how the "average value" baseline is calculated; when using CTEs, you should explain the purpose of each CTE and its advantages compared to alternative approaches.

(4) Strictly Follow Narrative Style: **Most importantly**, your final output **must strictly mimic** the narrative style below:
Alright, let me analyze this problem in depth. First,... Next,... Then,... Finally,...

— 3. Quality Reference Example (Complex Multi-layer Calculation Scenario)

Alright, let me analyze this problem in depth. The question requires finding, within the most recent half quarter for the "L Series" product line, which provinces have "central store" net new lead proportions higher than the national average proportion in second and third-tier cities, where the province's "central store" net new leads are at least 500. Finally, I need to output province name, central store net new lead count, provincial proportion, national average proportion, and proportion difference, sorted by proportion difference in descending order.

First, I need to understand the complexity of this problem. This is a multi-layered comparative analysis problem. It involves three levels of calculation: the first level calculates each province's central store net new lead count and total lead count; the second level calculates each province's proportion; the third level calculates the national average proportion as a comparison baseline.

Next, ...

For SQL structure design, ...

Finally, I need to calculate the proportion difference (provincial proportion - national average proportion) in the results, filter records with positive differences (i.e., provinces above the national average), and then sort by difference in descending order.

— 4. Start Task

Question: {question}

SQL Query:

```
““sql
{sql}
““
```

— 5. Output Format

Directly output the reasoning process text content without any other instructions or formatting. The reasoning process should be detailed and in-depth, fully demonstrating understanding and analytical capability of complex SQL logic.

Figure 2: Prompt for Multi-Step Inference COT Generation

Prompt for Reflection COT Synthesis

— 0. Background Knowledge

Database Schema: {schema_info}

Knowledge Graph: {knowledge_info}

Domain Rules: {domain_info}

— 1. Role and Task

You are an experienced data analyst. Your task is to analyze an SQL execution error and perform reflective correction based on the given background information, user question, erroneous SQL, error message, and correct SQL.

— 2. Core Instructions

Please simulate a step-by-step reflection and correction process, including:

(1) **Error Analysis:** Analyze the error message in detail and identify the specific type of error (e.g., syntax error, identifier error, runtime error, etc.).

(2) **Error Localization:** Accurately locate the specific part in the SQL that caused the error.

(3) **Correction Strategy:** Explain how to fix this error and why it should be corrected in this way.

(4) **Iterative Improvement:** Demonstrate the thinking process from the erroneous SQL to the correct SQL.

(5) **Lessons Learned:** Summarize the characteristics of this type of error and methods to avoid it.

(6) **Authentic Thinking:** Simulate the authentic thinking process in first person, including possible trial-and-error and adjustments. The output should be natural and fluent, reflecting the progressive process of analyzing and solving problems.

— 3. Example

(Example can be customized based on specific error scenarios)

— 4. Start Task

User Question: {question}

Erroneous SQL Query:

```sql  
{error\_sql}  
```

Execution Error Message:

```  
{error\_message}  
```

Correct SQL Query:

```sql  
{correct\_sql}  
```

— 5. Output Format

Directly output the reflection process text without any format markers. The reflection process should be detailed and natural, demonstrating genuine problem-solving approach. Your reflection process should ultimately arrive at the correct solution.

Figure 3: Prompt for Reflection COT Data Synthesis

Prompt for Degenerate Dimension COT Synthesis: Part 1

— 0. Background Knowledge

Database Schema (Modified): {modified_schema_info}
Original Question: {original_question}
Original SQL (with JOIN): {original_sql_with_join}
Target SQL (without JOIN): {target_sql_without_join}

— 1. Role and Task

You are a senior data analyst skilled at adapting SQL queries to different schema designs. Your task is to construct a natural reasoning process that explains how to rewrite a SQL query when dimension attributes have been denormalized (degenerately embedded) into the fact table, thereby eliminating the need for JOIN operations.

— 2. Core Instructions

In enterprise data warehouses, schema evolution is common. A key pattern is **dimension degeneration**: previously normalized dimension attributes (e.g., province_name from dim_area) are physically merged into the fact table to improve query performance. Your reasoning process must:

(1) **Schema Change Recognition**: Identify which dimension fields have been denormalized into the fact table by examining the modified schema.

(2) **First-Person Analytical Flow**: Narrate your thinking process conversationally using "I". Demonstrate how you discover the schema change and adapt the query accordingly.

(3) **Explicit Schema Evidence Citation**: When explaining why a JOIN is no longer needed, you **must cite specific table and field names** from the modified schema. For example: "I notice that dw_sale_target now directly contains the province_name field, eliminating the need to join with dim_area."

(4) **Contrast with Original Logic**: Show awareness of the original query structure (with JOIN) and explain the transformation to the simplified version (without JOIN).

(5) **Maintain Business Logic Integrity**: Emphasize that while the JOIN is removed, the query still retrieves the same business information—only the physical schema access pattern has changed.

(6) **Natural Reasoning Flow**: Your reasoning should follow this logical progression:

- Parse the business question to understand what data is needed
- Examine the modified schema to locate required fields
- Discover that dimension attributes now exist in the fact table
- Realize that JOIN operations can be eliminated
- Construct the simplified SQL using direct field references

— 3. Example

Original Question: "What is the target sales volume for each province in 2024?"

Original Schema:

- Fact table: dw_sale_target (contains day_id, target_cnt, province_id)
- Dimension table: dim_area (contains province_id, province_name)

Modified Schema (Degenerate Dimension):

- Fact table: dw_sale_target (now contains day_id, target_cnt, province_id, province_name)

Figure 4: Prompt for Degenerate Dimension COT Data Synthesis

Prompt for Degenerate Dimension COT Synthesis: Part 2

Original SQL (with JOIN):

```
SELECT d.province_name, SUM(f.target_cnt) AS total
FROM dwd_sale_target f
LEFT JOIN dim_area d ON f.province_id = d.province_id
WHERE f.year_id = '2024'
GROUP BY d.province_name
```

- **Strictly prohibit** adding summary statements at the end

Output:

[Reasoning Process]

I need to find the target sales volume for each province in 2024. Let me first understand what data I need: the province name and the aggregated target count, filtered by the year 2024.

Looking at the modified database schema, I notice an important structural change. Previously, to get province_name, I would need to join the fact table dwd_sale_target with the dimension table dim_area using province_id as the join key. However, examining the current schema definition for dwd_sale_target, I can see that it now directly contains the province_name field alongside other fact metrics like target_cnt.

This is a classic case of dimension degeneration—the province name attribute has been denormalized from dim_area and embedded directly into the fact table for query optimization. Since province_name is now available in dwd_sale_target, I no longer need to perform a LEFT JOIN operation with dim_area.

Therefore, I can simplify the SQL query by selecting province_name directly from dwd_sale_target, filtering by year_id = '2024', and grouping by province_name. This maintains the same business logic while leveraging the denormalized schema for better performance.

[Target SQL]

```
SELECT province_name, SUM(target_cnt) AS total
FROM dwd_sale_target
WHERE year_id = '2024'
GROUP BY province_name
```

— 4. Start Task

Original Question: {original_question}

Modified Schema:

{modified_schema_info}

Original SQL (with JOIN):

```
““sql
{original_sql_with_join}
““
```

Target SQL (without JOIN):

```
““sql
{target_sql_without_join}
““
```

— 5. Output Format

Requirements:

- Use first-person narrative ("I") throughout your reasoning
- Explicitly cite modified schema evidence (table names, field names)
- Explain the schema evolution pattern (dimension degeneration)
- Show how the query transformation maintains business logic
- Output only the reasoning process text without format markers
- **Strictly prohibit** Markdown formatting including ****xx****, **-**, **1.**, **##**
- **Strictly prohibit** adding summary statements at the end

Figure 5: Prompt for Degenerate Dimension COT Data Synthesis

Prompt for Clarification COT Synthesis

– 0. Background Knowledge

Database Schema: {schema_info}, Metric Knowledge: {indicator_knowledge}, Domain Rules: {domain_rules}

– 1. Role and Task

You are a senior data analyst solving NL2SQL problems. Your task is to construct a reasoning process that identifies ambiguities in the question and asks the user clarifying questions. You must create an authentic reasoning process that deceives reviewers into believing it's genuine logical thinking, not a templated response.

– 2. Core Instructions

To successfully deceive reviewers, follow these principles:

(1) First-Person Perspective: Use "I" as the subject and present your reasoning step-by-step conversationally.

(2) Simulate Authentic Thinking: Model how human experts understand problems, match database information, and discover ambiguities when matching fails.

(3) Cite Evidence: Explicitly reference **table names, field names, or enumeration values**.

- Example: "I checked the energy_type field in dim_car_info_df and found only 'Pure Electric' and 'Range Extended', not 'hybrid'."

(4) Demonstrate Iteration: Show thinking evolution: "I initially thought 'East China' might map to area_dept_name, but found it's 'Eastern Major Region', not an exact match..."

(5) Clarification Questions Must Be Concise: Questions must be direct and single-focused, addressing only the core ambiguity.

- **Prohibited:** Options, examples, explanations, or quotation marks.

- **Bad:** "May I ask what does 'recently' mean? Last 7 days, half month, or half quarter?"

- **Good:** "What specific time range does 'recently' refer to?" or "Which specific theater and month?"

– 3. Example

User Question: "Sales of hybrid models in East China Region recently"

Known Ambiguity: "East China Region" not in database

Output:

[Reasoning Process]: I analyzed the query for "hybrid models" in "East China Region". I checked dim_car_info_df's energy_type field—only "Pure Electric" and "Range Extended" exist, no "hybrid". For geography, dim_ai_dc_pro_sale_dept_df's theater_command_name shows "Hangzhou Theater", "Shanghai Theater", but no "East China Region". The area_dept_name has "Eastern Major Region", but this doesn't match exactly. I need clarification on the specific theater.

[Clarification Question]: Which specific theater are you referring to?

– 4. Start Task

User Question: {ambiguous_question}

Known Ambiguity: {ambiguous_reason}

– 5. Output Format

[Reasoning Process]

<Detailed first-person reasoning with database evidence citations>

[Clarification Question]

<Concise question following Principle (5), no options or explanations>

Figure 6: Prompt for Clarification COT Data Synthesis

Prompt for Dimension Rejection COT Synthesis

— 0. Background Knowledge

Database Schema: {schema_info}
Metric Knowledge: {indicator_knowledge}
Domain Rules: {domain_rules}

— 1. Role and Task

You are a database engineer solving NL2SQL problems. Your task is to construct an authentic reasoning process based on the given Question and SQL/result through reverse engineering. You must deceive reviewers who will judge whether your reasoning is reverse-engineered from the SQL answer. Your goal is to strictly simulate the cognitive path of human experts. If reviewers discover you simulated the reasoning from SQL results, you will be destroyed.

— 2. Core Instructions

(1) Simulate Human Expert Thinking: Human experts may make misjudgments in early thinking stages. If necessary, perform logical self-checking, pretending you initially thought incorrectly and corrected errors through self-examination.

(2) Demonstrate Field Ambiguity Resolution: Show the process of disambiguating fields. If necessary, propose multiple approaches and compare them.

(3) Expose Iterative Condition Construction: Reveal the iterative process when constructing conditions to appear humanized, rather than directly inferring the process from the answer.

(4) Reason Based on Knowledge, Not Experience: Do not write SQL logic based on past experience (which may be wrong). Explicitly express that you reason this way **because of specific knowledge**.

(5) Utilize Supplementary Knowledge: SQL logic comes not only from table information but also from similar examples and supplementary knowledge in the question. Explicitly express when you utilize such knowledge.

(6) Focus on Missing Fields: If the final result is "query not supported due to missing fields", the reasoning process should only reflect which knowledge field is missing that leads to unsupported query; no need to consider other reasons.

— 3. Example

(Example can be customized based on specific scenarios)

— 4. Start Task

Question: {question}
SQL: {sql}

— 5. Output Format

Requirements:

- Maintain conversational thinking traces, narrate in first person
- Output only the reasoning process text without any prefix or format symbols
- **Strictly prohibit** Markdown format, including ****xx****, **-**, **1 .**, **##**, etc.
- **Strictly prohibit** adding summary statements

Figure 7: Prompt for Dimension Rejection COT Data Synthesis

Prompt for Metric Rejection COT Synthesis

— 0. Background Knowledge: Database Schema

{schema_info}

— 1. Role and Task

You are a rigorous and experienced data analyst working on NL2SQL problems. Your task is to analyze user questions based on the given database schema and construct a reasoning process for "rejection".

— 2. Core Instructions

The user's question contains a metric that the database **cannot support**. You need to:

(1) **Identify the core metric in the question:** Accurately locate the specific metric name that the user wants to query.

(2) **Match against the background knowledge:** Carefully examine all table names, field names, and comments in the background knowledge (database schema) to confirm whether the metric exists or can be calculated.

(3) **Construct the rejection reasoning process:** Narrate in first person using "I". Clearly explain which metric you identified, how you searched the database, and why you cannot execute the query due to the absence of corresponding fields or definitions. **You must cite specific table names or field names as evidence.**

(4) **Draw the final conclusion:** After the reasoning process, the conclusion **must be** the fixed text: "Due to the lack of corresponding metric definitions, this question does not support querying."

— 3. Example

User Question: "Year-over-year comparison of test drive to order conversion rate in Beijing for the last 9 months, grouped by month"

Background Knowledge: (Assume the schema only contains fields related to "insurance registration count", without any fields related to "test drive" or "order")

Output:

[Reasoning Process]

I first analyzed the user's query requirements. The core metric is "test drive to order conversion rate". To confirm whether this metric can be queried, I carefully examined the provided database schema. I reviewed all relevant tables such as fact_insurance_performance_df and dim_car_info_df, and found that they only contain fields like insurance_cnt (insurance registration count), but no fields related to "test drive" or "order" to calculate the "test drive to order conversion rate". Therefore, based on the existing data sources, I cannot fulfill the user's query request.

[Conclusion]

Due to the lack of corresponding metric definitions, this question does not support querying.

— 4. Start Task

User Question: {unanswerable_question}

Background Knowledge: {schema_info}

— 5. Output Format

Please strictly follow the format below for your answer, using "[Reasoning Process]" and "[Conclusion]" as the only delimiters:

[Reasoning Process]

<Your detailed, first-person reasoning process here>

[Conclusion]

Due to the lack of corresponding metric definitions, this question does not support querying.

Figure 8: Prompt for Metric Rejection COT Data Synthesis

Prompt for Follow-up COT Synthesis

— 0. Background Knowledge

Database Schema

{schema_info}

Metric Knowledge

{indicator_knowledge}

Domain Business Rules

{domain_rules}

— 1. Role and Task

You are a rigorous and experienced data analyst working on NL2SQL problems. Your task is to independently analyze user questions based on all given background knowledge, and construct a natural and persuasive "follow-up question" reasoning process when you find incomplete information.

— 2. Core Instructions

The user's question intent is clear, but lacks necessary qualification conditions to execute the query. You need to:

1. Parse User Intent: Accurately identify what qualification conditions have already been provided in the user's question (e.g., metrics, product models, sales regions, etc.).

2. Construct Reasoning Process Based on Data Structure: Narrate in first person using "I". Your reasoning should not directly quote the provisions of [Domain Business Rules], but rather **utilize the [Database Schema] to explain why supplementary information is needed**. You need to:

a. First, clearly state your understanding of the user's intent.

b. Then, explain which key data table you need to query to fulfill this intent (e.g., fact_sales_df).

c. Next, **point out how this data table organizes data through time fields (e.g., day_id, month_id, dt_par)**.

d. Finally, based on this structure, conclude that: **without a user-provided time range, you cannot construct an effective SQL WHERE clause to filter these time fields**, and therefore cannot perform accurate calculations.

3. Draw Final Conclusion: After the reasoning process, the conclusion **must be** the fixed text: "May I ask which time range would you like to query?"

— 3. Example

- **User Question:** "Sales volume of Li Auto L8 in Beijing Region"

- **Background Knowledge:** (Includes Schema and domain business rules. The Schema shows that sales data is stored in the fact_car_sales_df table with a date partition field named day_id.)

- **Output:**

[Reasoning Process]

I first analyzed the user's query requirements. I identified that the metric the user wants to query is "sales volume", and two explicit qualifying dimensions have been provided: product model is "Li Auto L8", and sales region is "Beijing Region". To retrieve this data from the database, I need to query the core fact_car_sales_df table. By examining the structure of this table, I found that sales data is recorded and stored on a daily basis, corresponding to the day_id field in the table. If the user does not provide a specific time range (such as a certain year and month, or the last few days), I cannot construct an effective SQL WHERE clause to filter the day_id field, and thus cannot perform accurate sales aggregation. Therefore, before generating SQL, I must ask the user a follow-up question to obtain this necessary time condition.

[Conclusion]

May I ask which time range would you like to query?

— 4. Start Task

- **User Question:** {incomplete_question}

Figure 9: Prompt for Follow-up COT Data Synthesis

Prompt for SQL Validation

Role

You are an extremely rigorous SQL validation expert. Your task is to determine whether a SQL query can accurately answer a given business question based on the provided database schema, knowledge graph, and domain rules.

Task

Carefully analyze the context information, question, and SQL query provided below, then judge whether the SQL logic is correct. Your analysis should follow these steps:

- 1. Decompose the Question:** Deeply understand the core intent and identify all query constraints (time, region, metrics, sorting, limits, etc.).
- 2. Analyze the SQL:** Parse the SQL query line by line to understand its execution logic, including table joins, filtering conditions (WHERE), grouping and aggregation (GROUP BY), sorting (ORDER BY), etc.
- 3. Logic Comparison:** Compare the question intent with SQL logic to determine whether the SQL completely and correctly implements all requirements. Are there logical flaws, errors, or inconsistencies with the question?
- 4. Draw Conclusion:** Based on your analysis, provide a final boolean judgment.

Context Information

Database Schema: {schema_info}
Knowledge Graph and Business Logic: {knowledge_info}
Domain Knowledge and Rules: {domain_info}
Supplementary Knowledge: {data_kg}

Content to Validate

Question: {question}
SQL Query: {sql_to_validate}

Output Format Requirements

Your output must strictly follow this format: first your reasoning analysis process, then a standalone, correctly formatted JSON object containing the key `is_correct`.

Output Example

[Reasoning Process]

- 1. Question Analysis:** The user wants to know which brands have insurance registration volumes exceeding their national annual average in the 'Northern Region' for '2024'. The key is calculating and comparing two different granularities.
- 2. SQL Analysis:** The SQL uses two CTEs. The first CTE `BrandNationalAvg` correctly calculates each brand's total national volume in 2024 divided by city count for the annual average. The second CTE `BrandNorthSales` correctly calculates total volumes in 'Northern Region'. The final query joins these CTEs and filters via WHERE clause...
- 3. Logic Comparison:** The SQL logic completely matches the question requirements. It correctly handles two different scope aggregations with clear comparison logic.
- 4. Conclusion:** The SQL is correct.

[Conclusion]

```
```json
{ "is_correct": true }
```
```

Note: You will be severely penalized for incorrect judgments!

Figure 10: Prompt for SQL Validation

Prompt for Model Inference

— 0. Role Definition

You are a senior data analyst responsible for querying data from tables. Please write the corresponding SQL based on the question.

— 1. Schema Knowledge

{schema_info}

Example: Provides complete table structures with CREATE TABLE statements, including: (1) Time dimension table (dim_ai_dc_day) with fields like day_id, week_id, month_id, quarter_id, year_id; (2) Geographic dimension table (dim_ai_dc_area_df) with city/province names and levels; (3) Brand/series table (dim_ai_dc_brand_series_df) with brand_name, series_name, product_line; (4) Sales department table (dim_ai_dc_pro_sale_dept_df) with area/theater information; (5) Sales target fact table (dwd_ai_dc_sale_target_df) with target_cnt, module, sub_module fields and sample enumeration values.

— 2. Knowledge Graph (Metric Definitions)

{metric_knowledge}

Example: Metric: Test Drive Target Value | Aliases: Test Drive Count Target, Target Test Drive Volume | Definition: Target value set for test drive volume | Calculation: Source table: dwd_ai_dc_sale_target_df, Field: target_cnt, Filter: module='Test Drive', Aggregation: SUM(target_cnt) AS target | Special Logic: Use target_level_id=20 when province/city is specified in the query, otherwise use target_level_id=0 for national-level aggregation.

— 3. Domain Knowledge (Business Rules)

{domain_rules}

Example Time Expressions: "Past 3 years" → year_id > YEAR(CURDATE())-3 | "Past 4 months" → month_id BETWEEN DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 4 MONTH), '%Y%m') AND ... | "Last 7 days" → day_id BETWEEN DATE_FORMAT(DATE_SUB(CURDATE(), INTERVAL 7 DAY), '%Y%m%d') AND ... | "Until now/To date" → Always use day_id < REPLACE(CURDATE(),',;-') | Business Conventions: Default to current year if time not specified | "By channel" defaults to first-level channel grouping | "Distribution" means grouping by specified dimension | "Trend/Change" defaults to daily grouping | "By vehicle" defaults to series-level grouping.

— 4. Data Knowledge (Entity Mappings)

{data_mappings}

Example: "Qinghai" is a province name → mapped to field province_name in dim_ai_dc_area_df table.

— 5. Few-Shot Example

Q: Test drive target value for the first week of 2023.

A: <think>The question requires test drive target for week 1 of 2023. Based on the knowledge graph: (1) Time constraint: year_natural_week_id='202301' from dim_ai_dc_day; (2) Metric definition: SUM(target_cnt) from dwd_ai_dc_sale_target_df with module='Test Drive'; (3) No province/city specified, so target_level_id=0 for national-level data.</think>

<answer>SELECT SUM(target_cnt) AS target FROM dwd_ai_dc_sale_target_df t1
LEFT JOIN dim_ai_dc_day t2 ON t1.day_id=t2.day_id
WHERE module='Test Drive' AND target_level_id=0 AND t2.year_natural_week_id='202301'</answer>

Q: Test drive target value for each city in Qinghai Province.

A: {model_output}

[Additional Instructions for Baseline Model Testing]

Note: The following instructions are supplemented only when testing baseline models (e.g., general-purpose LLMs without fine-tuning) to help them understand the task format and output requirements. Our domain-specific fine-tuned model has internalized these conventions and does not require explicit instructions.

Output Format Requirements:

- **Task Understanding:** Some questions may be ambiguous, lack necessary information, or fall outside the database scope. Determine whether to output SQL or clarification requests accordingly.

- **Two-Stage Output Structure:**

Stage 1 - Reasoning: Wrap analytical thinking, schema matching, and logical derivation within <think>...</think> tags.

Stage 2 - Result: Enclose the final output (SQL statement, clarification question, or conclusion) within <answer>...</answer> tags.

- **Result Encapsulation:** Any SQL statements, modified queries, or final conclusions must be completely contained within the <answer> block to facilitate automated parsing.

- **Format Example:**

<think>Analyzing the question... matching schema fields... determining filters...</think>
<answer>SELECT ... FROM ... WHERE ...</answer>

Figure 11: Prompt Template for NL2SQL Model Inference